

SQL VALIDATION LAYER

1 - Validate Total Rows and customer Churn

```
MariaDB [(none)]> show databases;
```

Database
customer_churn
information_schema
iot_analytics
mysql
performance_schema
sys
test

```
7 rows in set (0.004 sec)
```

```
MariaDB [(none)]> use customer_churn;
```

```
Database changed
```

```
MariaDB [customer_churn]> select count(*) as total_rows,  
-> count(distinct customer_id) as unique_customers from  
-> customer_churn;
```

total_rows	unique_customers
7043	7043

```
1 row in set (0.026 sec)
```

```
MariaDB [customer_churn]> |
```

2 - Validate NULL Values

```
MariaDB [customer_churn]> select count(*) as total_rows ,  
-> sum(customer_id IS NULL) as customer_id_nulls,  
-> sum(gender IS NULL) as gender_nulls,  
-> sum(total_charges is null) as total_charges_null ,  
-> sum(monthly_charges is null) as monthly_charges_null ,  
-> sum(churn_flag is null) as churn_flag_nulls  
-> from customer_churn;
```

total_rows	customer_id_nulls	gender_nulls	total_charges_null	monthly_charges_null	churn_flag_nulls
7043	0	0	11	0	0

```
1 row in set (0.013 sec)
```

3 - validate the Value Mapper transformations

```
MariaDB [customer_churn]> SELECT
-> MIN(has_partner) AS min_has_partner,
-> MAX(has_partner) AS max_has_partner,
-> MIN(has_dependents) AS min_has_dependents,
-> MAX(has_dependents) AS max_has_dependents,
-> MIN(phone_service) AS min_phone_service,
-> MAX(phone_service) AS max_phone_service,
-> MIN(multiple_lines) AS min_multiple_lines,
-> MAX(multiple_lines) AS max_multiple_lines,
-> MIN(churn_flag) AS min_churn_flag,
-> MAX(churn_flag) AS max_churn_flag
-> FROM customer_churn;
```

min_has_partner	max_has_partner	min_has_dependents	max_has_dependents	min_phone_service	max_phone_service	min_multiple_lines	max_multiple_lines	min_churn_flag	max_churn_flag
0	1	0	1	0	1	0	1	0	1

1 row in set (0.017 sec)

As have to check that all boolean type values are mapped to binary form zero or one.

4 - validate categorical fields

Validate Internet Services

```
MariaDB [customer_churn]> SELECT internet_service, COUNT(*) AS row_count
-> FROM customer_churn
-> GROUP BY internet_service
-> ORDER BY internet_service;
```

internet_service	row_count
DSL	2421
Fiber optic	3096
No	1526

3 rows in set (0.013 sec)

Validate contract_type

```
MariaDB [customer_churn]> SELECT contract_type, COUNT(*) AS row_count
-> FROM customer_churn
-> GROUP BY contract_type
-> ORDER BY contract_type;
```

contract_type	row_count
Month-to-month	3875
One year	1473
Two year	1695

3 rows in set (0.009 sec)

Validate Payment method

```
MariaDB [customer_churn]> SELECT payment_method, COUNT(*) AS row_count
-> FROM customer_churn
-> GROUP BY payment_method
-> ORDER BY payment_method;
```

payment_method	row_count
Bank transfer (automatic)	1544
Credit card (automatic)	1522
Electronic check	2365
Mailed check	1612

4 rows in set (0.020 sec)

```
MariaDB [customer_churn]> |
```

5 - Customer Churn summary

```
MariaDB [customer_churn]> select churn_flag , count(*) as customer_count from customer_churn
-> group by churn_flag
-> order by churn_flag;
```

churn_flag	customer_count
0	5174
1	1869

2 rows in set (0.161 sec)

```
MariaDB [customer_churn]>
```

6 - Churn Rate

```
MariaDB [customer_churn]> SELECT
-> COUNT(*) AS total_customers,
-> SUM(churn_flag = 1) AS churned_customers,
-> SUM(churn_flag = 0) AS retained_customers,
-> ROUND(SUM(churn_flag = 1) * 100.0 / COUNT(*), 2) AS churn_rate_percent
-> FROM customer_churn;
```

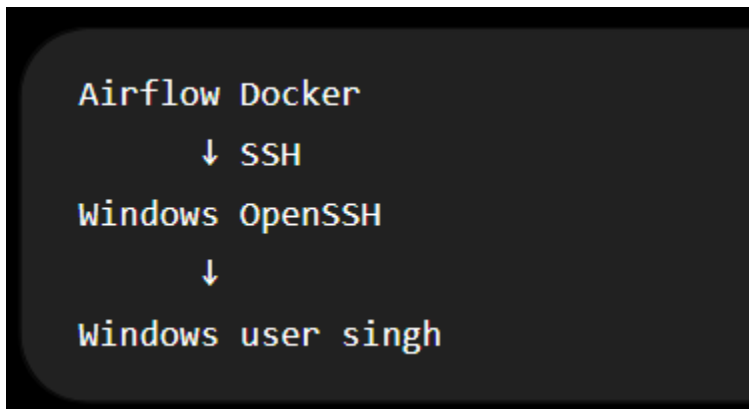
total_customers	churned_customers	retained_customers	churn_rate_percent
7043	1869	5174	26.54

1 row in set (0.023 sec)

Q1 - How is Airflow Triggering PDI?

Ans - Airflow is running on linux and is connected with windows with the help of openssh.

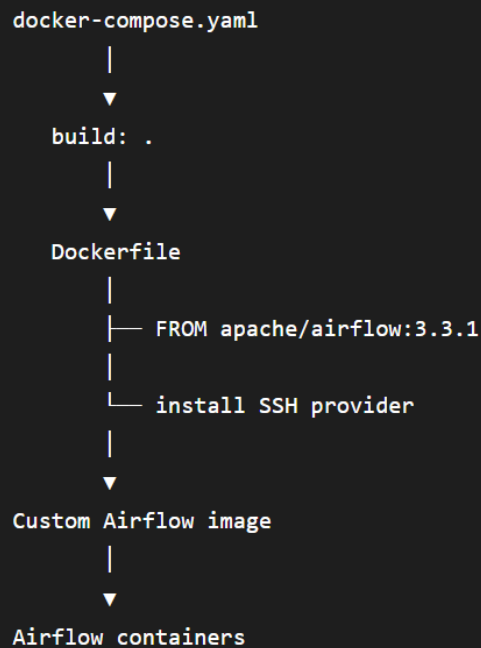
- Downloaded and verified open ssh.
- Made connection of open ssh from airflow to windows.



Now we're telling Docker Compose:

Don't simply use the official image. Build an image using the `Dockerfile` in this folder.

So the flow becomes:



What we've accomplished

```
Dockerfile
↓
apache/airflow:3.3.1
+
apache-airflow-providers-ssh 6.0.1
↓
Custom Airflow image
↓
Airflow Worker
↓
SSHOperator available ✅
```



And earlier we already verified:

```
Airflow Worker
↓ SSH
Windows OpenSSH
↓
Windows user: singh
✅
```



So the infrastructure required for the Airflow → Pentaho trigger is now ready.